Implementation Plan: Tastelet

Oct 11, 2026 · @Billy

Start with a 30-minute setup today, then build Phases 1-5 in Colab notebooks before touching the app; each phase lists its steps and a done-when check.

Where to start: setup (about 30 minutes, today)

Do these five things before Phase 1 so nothing blocks you tomorrow.

1. Create a GitHub repository named `tastelet` with a Python `.gitignore`.
2. Create the folders below and commit them.
3. Make a free TMDB account and request an API key (Settings, then API); store it as an environment variable, never in code.
4. Download `ml-latest-small.zip` from grouplens.org and unzip it into `data/raw/`.
5. Open Google Colab, create the first notebook, and confirm you can read `ratings.csv` with pandas.

```
tastelet/
  data/
    raw/            # MovieLens files (not committed)
    processed/      # splits, mappings, movies_meta.json
  notebooks/
    01_data_prep.ipynb
    02_baselines.ipynb
    03_model.ipynb
    04_foldin_personas.ipynb
  backend/
    app/            # main.py, model.py, recommend.py
    artifacts/      # ncf_model.pt, centroids.json, personas.json,
movies_meta.json
    requirements.txt
  frontend/         # Vite + React app
  README.md
```

Working rules

- Do Phases 1-5 in notebooks first, then move the finished code into `backend/app/`.
- Fix the random seed to 42 everywhere (`numpy`, `torch`, `train_test_split`, K-means).

- Commit at the end of each phase with a message like "phase 3: NCF trained".
- Write results into a `results.csv` as you go, so the report is mostly copy and paste.

Phase 1: Data prep (Oct 12-13, notebook 01)

Goal: turn raw MovieLens files into clean, indexed splits and a metadata file with posters.

1. **Load** `ratings.csv` (userId, movieId, rating, timestamp), `movies.csv` (movieId, title, genres) and `links.csv` (movieId, imdbId, tmdbId) with pandas.
2. **Clean:** drop movies with no `tmdbId` and movies with fewer than 5 ratings; extract the release year from the title, e.g. "Heat (1995)".
3. **Fetch TMDB metadata** for the remaining movies: call `https://api.themoviedb.org/3/movie/{tmdbId}?api_key=...`, keep `poster_path` and `overview`, and build the poster URL as `https://image.tmdb.org/t/p/w342` + `poster_path`. Sleep 0.05 s between calls and cache results to a local JSON so you only fetch once.
4. **Drop movies without a poster**, then drop their ratings. From here on, every movie in the model has a poster.
5. **Re-index** users and movies to 0..N-1 (`user2idx`, `movie2idx`) and save both mappings as JSON.
6. **Split** ratings 80/10/10 into train, validation and test with `train_test_split(random_state=42)`; save as CSV or parquet.
7. **Build home grid lists** and save them inside `movies_meta.json`:
   - all-time: at least 50 ratings, sorted by average rating
   - popular: sorted by number of ratings
   - trending: TMDB `/trending/movie/week`, filtered to movies in your data (if fewer than 12 remain, fill with popular)

Outputs: `ratings_train/val/test`, `user2idx.json`, `movie2idx.json`, `movies_meta.json` (id, title, year, genres, poster, overview, plus the three lists).

Done when: the three splits load, no movie in the data lacks a poster, and the three home lists each contain at least 12 movies.

Phase 2: Baselines (Oct 13-14, notebook 02)

Goal: build the evaluation code and three simple methods the neural network must beat.

1. **Write the evaluation helpers first** and reuse them for every model:
   - `rmse(model_predict, test_df)`: square root of mean squared error on test ratings.
   - `hit_at_10(score_fn, train_df, test_df)`: for each test user, rank all movies the user did not rate in train, then count a hit if at least one test movie rated 4.0 or higher appears in the top 10. Report the share of users with a hit.
2. **Global mean:** predict the train mean rating for everything. This gives the RMSE floor to beat.
3. **Popularity:** rank movies by number of train ratings, the same list for every user. Hit@10 only (RMSE does not apply).
4. **Item-based KNN:**
   - Build a sparse user-by-movie matrix from train.
   - Compute cosine similarity between movie columns (`sklearn.metrics.pairwise.cosine_similarity`).
   - Predict a rating as the similarity-weighted average of the user's ratings on the 20 most similar movies; fall back to the global mean when none exist.
   - For ranking, score each unseen movie by its similarity to the user's liked movies.
5. **Record results** in `results.csv` with columns `model, rmse, hit_at_10`.

Done when: all three baselines have numbers in `results.csv`, and the same evaluation code will run unchanged on the neural model.

Phase 3: Neural model (Oct 14-17, notebook 03)

Goal: train the neural collaborative filtering model and compare it with the baselines.

1. **Dataset and loader:** a PyTorch `Dataset` returning `(user_idx, movie_idx, rating)`; `DataLoader` with batch size 256, shuffle on for train.
2. **Model:** user and movie embeddings of size 32, concatenated into an MLP of 64, 32 and 1 units with ReLU and dropout 0.2 (a small PyTorch module with two embedding layers and a sequential MLP).
3. **Training loop:**
   - MSE loss, Adam with learning rate 1e-3 and weight decay 1e-5.
   - Up to 20 epochs; after each, compute validation RMSE and save the weights when it improves.
   - Log train and validation loss per epoch for the report plot.

4. **Tune** one setting at a time: embedding size (16, 32, 64), dropout (0.1, 0.2, 0.3), learning rate (1e-3, 1e-2). Add each run to a small table.
5. **Evaluate the best model** on the test set with the same `rmse` and `hit_at_10` helpers, and append it to `results.csv`.
6. **Save** `ncf_model.pt` (state dict) together with `n_users`, `n_movies` and `dim` so the backend can rebuild the model.

Done when: the model is saved, loss curves are plotted, and the neural row in `results.csv` beats popularity on both metrics. If it ties KNN, keep going; the honest comparison is the result.

Phase 4: Cold-start fold-in (Oct 17-19, notebook 04)

Goal: given the movies a new user picked, fit a user vector with the trained model frozen, then rank all unseen movies.

1. **Add a method** to the NCF class, `score_vec(user_vec, movie_idx)`, that runs the same MLP but takes a user vector directly instead of a user index.
2. **Write `fold_in`:** freeze every weight, start from the average trained user vector, and optimize only that vector.
3. **Rank:** score all movies with the fitted vector, remove the picks, keep the top 20. Show the match score as predicted rating divided by 5, as a percentage, clipped to 0-100.
4. **Sanity test with hand-made users:** 5 animated family films, 5 horror films, 5 1990s crime dramas. Check that the recommendations fit each taste, and that different tastes give clearly different lists.
5. **Measure it for the report:** for 100 test users, use their 5 highest-rated train movies as picks, run `fold_in`, and compute Hit@10 against their test movies. Compare with the popularity baseline.

```
def fold_in(model, picks, global_mean, steps=100, lr=0.05, n_neg=20):
    model.eval()
    for p in model.parameters():
        p.requires_grad = False
    u = torch.nn.Parameter(model.user_emb.weight.mean(0,
keepdim=True).clone())
    opt = torch.optim.Adam([u], lr=lr)
    picked = set(picks)
    pool = [m for m in range(model.movie_emb.num_embeddings) if m not in
picked]
    movies = torch.tensor(list(picks) + random.sample(pool, n_neg))
    target = torch.tensor([5.0] * len(picks) + [global_mean] * n_neg)
    for _ in range(steps):
        opt.zero_grad()
        pred = model.score_vec(u.expand(len(movies), -1), movies)
        loss = ((pred - target) ** 2).mean() + 1e-3 * (u ** 2).sum()
        loss.backward()
        opt.step()
    return u.detach()
```

Why the extra 20 random movies: with only picks at 5.0, the vector can simply push every score up. Random unpicked movies pulled toward the average rating keep the taste specific. Test with and without them, and report which is better.

Done when: the three hand-made users get sensible, different lists, and the cold-start Hit@10 number is in `results.csv`.

Phase 5: Persona clustering (Oct 19-20, notebook 04)

Goal: group the trained user embeddings into about 8 taste clusters and give each a name.

1. **Extract embeddings:** `E = model.user_emb.weight.detach().numpy()` (610 rows), then L2-normalize each row so clusters follow taste direction rather than vector length.
2. **Try K from 6 to 12** with `KMeans(n_init=10, random_state=42)`; record inertia and silhouette score for each K. Silhouette will be low on 610 users in 32 dimensions, so use it only to rule out bad values and choose by how readable the clusters are.
3. **Profile each cluster** for the chosen K:
   - Count genres among the cluster's liked movies (train ratings of 4.0 or higher) and keep the top 3.

    - List the 5 movies with the highest like rate in the cluster, among movies with at least 10 raters there.
4. **Name each cluster by hand:** a persona name (e.g. "The Midnight Thriller Hunter") and a one-line description. If two clusters have the same top genres, merge them or pick another K.
5. **Save two files:** `centroids.json` (K normalized vectors) and `personas.json` (id, name, description, top genres).
6. **Runtime rule:** normalize the folded-in user vector and assign the nearest centroid by Euclidean distance (equivalent to cosine on normalized vectors).
7. **Check:** the three hand-made users from Phase 4 should land in different, sensible personas.

Done when: every cluster has a name and description, both JSON files are saved, and the persona table (name, top genres, example movies) is ready for the report.

Milestone: AI complete (Oct 20). The model, fold-in and personas are finished; everything after this is the application around them.

Phase 6: Genre sentence and FastAPI backend (Oct 20-22)

Goal: move the finished AI code into a backend that serves the three endpoints from the spec.

1. **Genre sentence:** split each pick's genres on `|`, count them, take the top two, and find the most common decade (`year // 10 * 10`). Fill the template "You gravitate toward {g1} and {g2}, especially films from the {decade}s." Break ties alphabetically so the same picks always give the same sentence.
2. **Backend files:** `app/model.py` (NCF class and loader), `app/recommend.py` (`fold_in`, ranking, persona assignment, genre sentence) and `app/main.py` (routes, CORS, startup).
3. **Artifacts:** copy `ncf_model.pt`, `centroids.json`, `personas.json` and `movies_meta.json` into `backend/artifacts/` and load them once at startup (FastAPI lifespan), not per request.
4. **Endpoints:** `GET /movies`, `POST /recommend` and `GET /health`. Define the request with a Pydantic model (`movie_ids: list[int]`, 5 to 10 items); an unknown movie ID returns a 422 with a plain message.

5. **Make it deterministic:** use a fixed-seed random generator inside `fold_in` (for the negative sample) so the same picks always return the same persona and list; this is acceptance criterion F4.
6. **CORS:** allow `http://localhost:5173` now (Vite's default), and add the deployed frontend URL in Phase 8.
7. **requirements.txt:** `fastapi`, `uvicorn`, `numpy`, and the CPU build of `torch` (much smaller than the default); pin versions.
8. **Test locally:** run `uvicorn app.main:app --reload`, open `/docs`, and call `/recommend` with the three hand-made users. Write three small tests: 20 recommendations returned, no pick appears in them, and the same picks give the same persona.

Done when: `/recommend` answers in under 3 seconds locally, and the three tests pass.

Phase 7: React frontend (Oct 22-26)

Goal: build the three screens from the spec on top of the working API.

1. **Create the app:** `npm create vite@latest frontend -- --template react`, then put `VITE_API_URL=http://localhost:8000` in `.env`.
2. **Components:**
   - `App`: holds `step` (home or result), `picks`, `result`, `loading` and `error`.
   - `MovieSection` and `MovieCard`: a titled poster grid; the card shows poster, title, year and a selected state.
   - `PickBar`: sticky bar with the "n/5" counter and a "See my results" button, disabled below 5 picks.
   - `ResultPage`: persona name as the large heading, description, genre sentence, a row of recommendation cards with match percentages, and a Start over button.
3. **API client (`api.js`):** `getMovies()` and `recommend(ids)` using `fetch`; show a loading message while waiting, and an error message with a retry button on failure.
4. **Wake the backend:** call `/health` as soon as the app loads, so a sleeping free server starts warming before the user finishes picking.
5. **Behavior rules:** tapping toggles a pick; a 10th pick is the last (show a small hint after that); Start over clears picks and scrolls back to the top.
6. **Layout:** CSS grid with `repeat(auto-fill, minmax(140px, 1fr))`; test at 360 px width; use `loading="lazy"` and `alt` text on posters, with a placeholder when an image fails.

7. **Footer attribution:** credit MovieLens (GroupLens) and TMDB, and include TMDB's required notice that the product uses the TMDB API but is not endorsed or certified by TMDB.

Build order: static home grid with fake data, then selection and the counter, then real API calls, then the result page, then polish and error states.

Done when: features F1 to F6 from the spec all pass their acceptance criteria against the local backend, on both phone width and desktop.

Phase 8: Deploy and test (Oct 26-27)

Goal: put both parts online and test the whole flow like a real user.

1. **Backend on Render:** create a Web Service from the repo with root directory `backend`, build command `pip install -r requirements.txt`, and start command `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. The model files are small, so commit them to the repo. Hugging Face Spaces (Docker) is the fallback if Render gives trouble.
2. **Frontend on Vercel:** import the repo with root directory `frontend`, the Vite preset, and the environment variable `VITE_API_URL` set to the Render URL.
3. **Update CORS** in the backend with the Vercel URL, then redeploy it.
4. **Test from a fresh browser window (and your phone):**
   - Home grid loads and shows three sections with posters.
   - 4 picks keep the button disabled; 5 to 10 picks work; Start over resets.
   - Three very different tastes (family animation, horror, crime drama) give different personas and lists.
   - Picked movies never appear in the recommendations; the response arrives in under 3 seconds once the server is awake.
   - With the backend asleep, the loading message appears instead of a crash.
5. **Prepare the demo:** open the site 5 minutes before presenting to wake the backend, record a screen video of the full flow as a backup, and keep the local version runnable.

Done when: the deployed site passes every check above and you have the backup recording.

Phase 9: Report and slides (Oct 27-31)

Goal: turn the notebook results into the course report and a short presentation.

1. **Export figures from the notebooks** as PNG files: loss curves, the K-means K comparison, and the persona table. Turn `results.csv` into the comparison table (global mean, popularity, item-KNN, neural net, and the cold-start row).
2. **Write the report** (AI parts only) in the order of the checklist in the spec: problem and why neural networks fit, dataset and split, results table, loss curves, fold-in explanation with 2-3 worked examples, persona table, limitations and future work.
3. **Limitations to state honestly:** new releases are not in the data, the dataset is small (610 users), there is no dislike signal, and fold-in uses synthetic 5.0 ratings for the picks.
4. **Slides (about 10):** problem, data, how the neural model works, baselines vs. neural net, cold start, personas, live demo, limitations, future work.
5. **Demo script (about 3 minutes):** pick 5 family-animation films and show the result, press Start over, pick 5 horror films and show how the persona and list change, then point to the results table.
6. **Finish the repository:** a README with setup steps in under 10 lines, then tag the final commit `v1.0`. Rehearse the demo twice.

Done when: every item in the spec's report, demo and definition-of-done checklists is ticked.